

Proyecto Fase 1 – EconoMarket

Documento de Requerimientos Detallados e Historias de Usuario

Versión 1.0

- Melvin Alexander Valencia Estrada / 202111556
- Erick Enrique González Chávez / 201900621
- Dilan Conaher Suy Miranda / 201801194
- Juan Sebastian Higueros de Leon / 201807344
- María Isabel Masaya Córdova / 201800565

1. Control de versiones

Versión	Fecha	Descripción	Responsable
1.0	2025	Creación inicial del documento de requerimientos del proyecto EconoMarket – Fase 1.	Equipo de Desarrollo
1.1	2025	Ajustes y validación de requerimientos conforme al alcance final de la Fase 1.	Equipo de Desarrollo

2. Descripción general de la aplicación

EconoMarket es una plataforma de comercio electrónico desarrollada bajo una arquitectura 100% basada en microservicios, cuyo objetivo es permitir a los usuarios navegar productos, gestionar un carrito de compras, realizar procesos de checkout y pagos, y recibir notificaciones relacionadas con su actividad dentro del sistema.

La aplicación cuenta con autenticación y autorización mediante JSON Web Tokens (JWT), integración con servicios externos como catálogos de otros equipos y servicios de conversión de moneda (FX), así como persistencia de datos en MS SQL Server. El sistema se ejecuta de forma reproducible tanto en entornos locales como en la nube utilizando Docker Compose, y cuenta con un pipeline de Integración Continua (CI) para asegurar la calidad del software.

3. Alcance de la versión 1.0

La versión 1.0 del proyecto EconoMarket contempla el desarrollo y despliegue de las funcionalidades base del dominio e-commerce, incluyendo:

- Gestión de usuarios y autenticación con JWT.
- Navegación y consulta de catálogo de productos.
- Carrito de compras persistente.
- Proceso de checkout y pagos (tarjeta, cartera o combinación).
- Gestión de órdenes.
- Envío de notificaciones por correo electrónico.
- Integración con catálogos de proveedores externos.
- Conversión de moneda mediante un servicio FX con caché y fallback.
- Ejecución del sistema mediante Docker Compose en entorno local y Google Cloud.
- Pipeline de CI con construcción de imágenes y ejecución de pruebas.

4. Modelo de datos

El sistema EconoMarket utiliza múltiples bases de datos independientes, alineadas con la arquitectura de microservicios. Cada microservicio administra su propia base de datos, garantizando bajo acoplamiento, escalabilidad y aislamiento de responsabilidades.

4.1 Base de datos: economarket

Tabla: users

Descripción:

Almacena la información de los usuarios del sistema, incluyendo clientes, proveedores y administradores. Esta tabla es utilizada por el microservicio de autenticación y autorización (Users/Auth Service).

Campo	Tipo	Descripción
id	INT (PK)	Identificador único del usuario
email	NVARCHAR(255)	Correo electrónico único
password_hash	NVARCHAR(255)	Contraseña cifrada
first_name	NVARCHAR(100)	Nombre del usuario
last_name	NVARCHAR(100)	Apellido del usuario
nit	NVARCHAR(20)	Número de identificación tributaria
phone	NVARCHAR(20)	Teléfono del usuario
role	NVARCHAR(50)	Rol del usuario (cliente, proveedor, admin)
created_at	DATETIME2	Fecha de creación
updated_at	DATETIME2	Fecha de última actualización

Índices:

- Índice único sobre el campo email.

Restricciones y reglas:

- El campo email debe ser único.
- El campo role solo permite los valores: cliente, proveedor, admin.
- La contraseña se almacena únicamente como hash.

4.2 Base de datos: OrdersDB

Tabla: orders

Descripción:

Registra las órdenes generadas por los clientes como resultado del proceso de checkout.

Estructura:

Campo	Tipo	Descripción
id	UNIQUEIDENTIFIER (PK)	Identificador único de la orden
user_id	INT	Usuario que realizó la compra
total	DECIMAL(10,2)	Total de la orden
status	VARCHAR(50)	Estado de la orden
payment_reference	VARCHAR(255)	Referencia del pago
created_at	DATETIME2	Fecha de creación
updated_at	DATETIME2	Fecha de actualización

Tabla: order_items**Descripción:**

Almacena el detalle de los productos asociados a cada orden.

Campo	Tipo	Descripción
id	UNIQUEIDENTIFIER (PK)	Identificador del ítem
order_id	UNIQUEIDENTIFIER (FK)	Orden asociada
product_id	INT	Identificador del producto
product_name	VARCHAR(255)	Nombre del producto
quantity	INT	Cantidad comprada
unit_price	DECIMAL(10,2)	Precio unitario
subtotal	DECIMAL(10,2)	Subtotal del ítem

Restricciones y reglas:

- Una orden puede tener uno o más productos.
- Eliminación en cascada cuando se elimina una orden.

4.3 Base de datos: paymentsDB

Tabla: wallet

Descripción:

Representa la cartera electrónica del usuario para pagos internos

Campo	Tipo	Descripción
walletId	UNIQUEIDENTIFIER (PK)	Identificador del wallet
usuarioId	INT	Usuario propietario
balance	DECIMAL(10,2)	Saldo disponible
moneda	VARCHAR(3)	Moneda del saldo
creado	DATETIME2	Fecha de creación
actualizado	DATETIME2	Fecha de actualización

Tabla: metodosPago

Descripción:

Almacena los métodos de pago registrados por los usuarios.

Campo	Tipo	Descripción
metodoId	UNIQUEIDENTIFIER (PK)	Identificador del método
usuarioId	INT	Usuario asociado
numero	VARCHAR(19)	Número del método
tipo	VARCHAR(20)	Tipo de método (tarjeta, etc.)
activo	BIT	Estado del método
creado	DATETIME2	Fecha de creación

Tabla: transaccion

Descripción:

Registra las transacciones de pago realizadas por los usuarios.

Campo	Tipo	Descripción
transaccionId	UNIQUEIDENTIFIER (PK)	Identificador de la transacción
usuarioid	INT	Usuario que paga
ordenId	UNIQUEIDENTIFIER	Orden asociada
total	DECIMAL(10,2)	Total pagado
metodoPago	UNIQUEIDENTIFIER	Método de pago utilizado
completado	DATETIME2	Fecha de finalización

Reglas de operación:

- El total pagado debe coincidir con la suma del pago por wallet y tarjeta.
- No se permite procesar pagos con saldo insuficiente en la wallet.

4.4 Base de datos: EconoMarketNotificacionesDB

Tablas: `notificacionProveedor`, `notificacionCliente`, `productoNotificacion`

Descripción:

Gestiona las notificaciones enviadas a proveedores y clientes por eventos del sistema.

Reglas principales:

- Notificación a proveedor cuando el stock es menor o igual a 10.
- Notificación al cliente al completar una compra o por recordatorio de carrito.

4.5 Base de datos: EconoMarketCartDB

Tabla: `carro`

Descripción:

Representa el carrito persistente del usuario.

Estructura:

- `id_carro` (PK)

- id_usuario
- estado
- check_out
- total
- created_at
- updated_at

Tabla: productoCarrito

Descripción:

Productos asociados al carrito del usuario.

Reglas de operación:

- Un carrito pertenece a un único usuario.
- Un carrito no puede modificarse después de finalizar el checkout.

4.6 Base de datos: CatalogServiceDB

Tabla: products

Descripción:

Almacena los productos disponibles en el catálogo interno.

Estructura:

- id (PK)
- sku (único)
- name
- description
- price
- stock
- category
- imageUrl
- providerId

- createdAt
- updatedAt

Restricciones y reglas:

- El SKU debe ser único.
- El stock no puede ser negativo.
- El precio visible al cliente incluye un markup del 10%.

5. Requerimientos estratégicos

RES-01

Implementar una plataforma e-commerce basada en una arquitectura 100% de microservicios que permita escalabilidad, mantenibilidad e interoperabilidad entre equipos de desarrollo.

RES-02

Garantizar la integración segura entre los distintos microservicios mediante autenticación y autorización basada en JWT.

RES-03

Asegurar que el sistema sea desplegable de forma reproducible tanto en entorno local como en la nube utilizando Docker Compose.

6. Requerimientos funcionales

RFN-01

El sistema debe permitir a los usuarios registrarse e iniciar sesión mediante credenciales válidas y obtener un token JWT.

RFN-02

El sistema debe permitir a los usuarios visualizar el catálogo de productos internos y externos.

RFN-03

El sistema debe permitir a los usuarios agregar, actualizar y eliminar productos del carrito de compras de forma persistente.

RFN-04

El sistema debe permitir realizar el proceso de checkout y generar una orden asociada al usuario autenticado.

RFN-05

El sistema debe permitir realizar pagos utilizando tarjeta, cartera electrónica o una combinación de ambos.

RFN-06

El sistema debe enviar notificaciones por correo electrónico a clientes y proveedores según eventos definidos.

7. Requerimientos no funcionales

RNF-01

El sistema debe garantizar la seguridad de los datos mediante el uso de autenticación y autorización con JWT.

RNF-02

El sistema debe mantener disponibilidad y tolerancia a fallos en integraciones externas mediante mecanismos de caché y fallback.

RNF-03

El sistema debe ser mantenible, permitiendo la actualización independiente de cada microservicio.

RNF-04

El sistema debe responder a las solicitudes del usuario en tiempos aceptables bajo condiciones normales de carga.

8. Requerimientos técnicos

RTC-01

Los microservicios backend deben estar desarrollados utilizando TypeScript, Python o Golang.

RTC-02

La comunicación interna entre microservicios debe realizarse mediante gRPC.

RTC-03

La persistencia de datos debe realizarse utilizando MS SQL Server como base de datos principal.

RTC-04

Redis debe utilizarse únicamente como mecanismo de caché y fallback para integraciones externas y servicio FX.

RTC-05

El sistema debe contar con un pipeline de Integración Continua (CI) que ejecute pruebas y construya imágenes Docker.

9. Restricciones

RTR-01

La orquestación del sistema en esta fase debe realizarse utilizando Docker Compose tanto en entorno local como en una máquina virtual en Google Cloud.

RTR-02

Redis no puede ser utilizado como base de datos principal ni como fuente de verdad del dominio.

RTR-03

Los servicios internos no deben exponer directamente sus bases de datos ni puertos sensibles al exterior.

10. Riesgos

RGO-01

Fallas en la disponibilidad de servicios externos (proveedores o FX) que afecten la experiencia del usuario.

RGO-02

Errores de configuración en la autenticación JWT que permitan accesos no autorizados.

RGO-03

Inconsistencias de datos provocadas por errores en la comunicación entre microservicios.

11. Reglas de negocio para sesiones de trabajo

RNG-01

El precio visible al cliente debe calcularse aplicando un markup del 10% sobre el precio base del proveedor sin mostrarse como un concepto separado.

RNG-02

El carrito de compras debe mantenerse persistente y asociado al usuario autenticado.

RNG-03

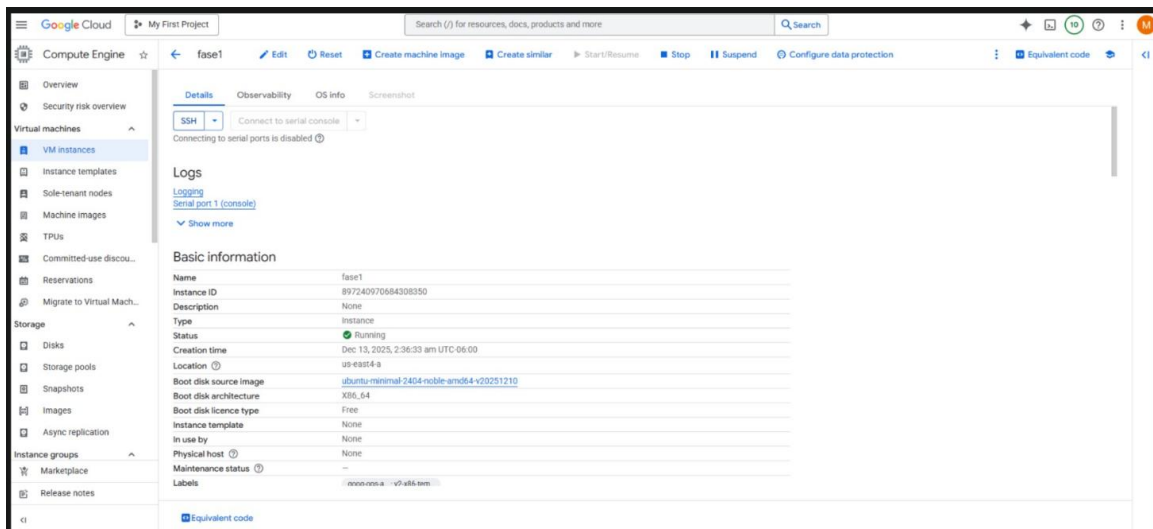
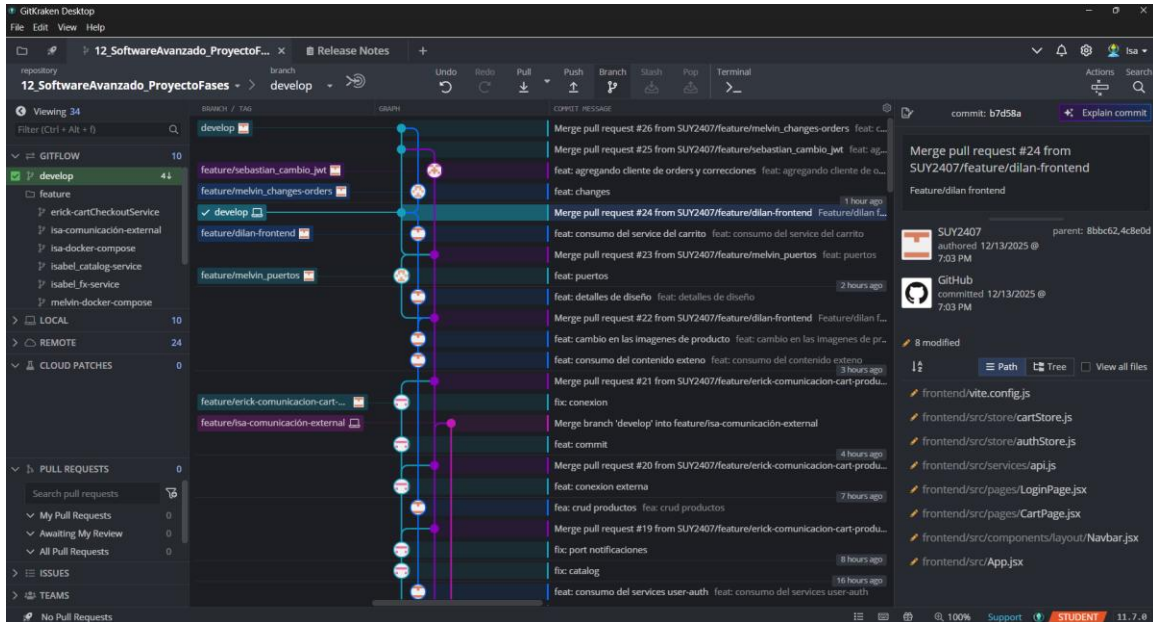
Una orden solo puede generarse si el proceso de pago se completa correctamente.

RNG-04

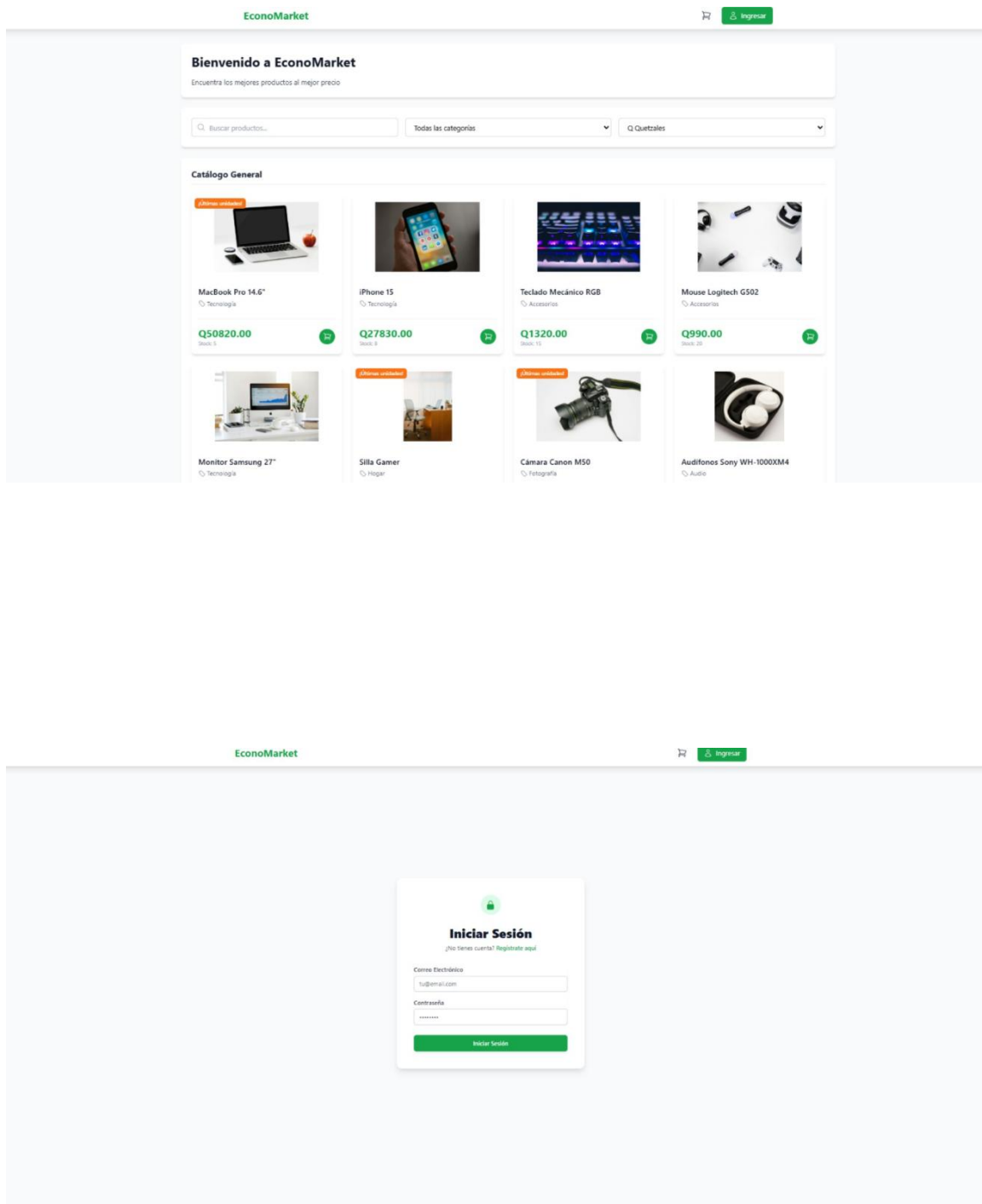
Las notificaciones deben enviarse únicamente cuando se cumplan las condiciones definidas para evitar envíos duplicados.

Proyecto Fase 1 - EconoMarket – Versión 1.0

ANEXOS



Proyecto Fase 1 - EconoMarket – Versión 1.0



Proyecto Fase 1 - EconoMarket – Versión 1.0

EconoMarket

 [Ingresar](#)



Crear Cuenta

Nombre

Apellido

Email

NIT

Teléfono

C.P.

Tipo de Cuenta

Cliente (Comprador) ▾

Contraseña

Confirmar Contraseña

[Registrarse](#)